



USC University of
Southern California

Software 3.0 and the Claw Phenomenon

CSCI 599 — Agentic AI

Lecture 1 / Week 1



What changed in two years

2024 — agents were demos

- AutoGPT, BabyAGI: impressive loops, brittle in production
- Hype outran what actually shipped

2026 — agents are infrastructure

- TELUS: 13,000+ custom AI solutions, 500,000+ hours saved
- Rakuten: a seven-hour Claude Code run on 12.5M lines, with occasional engineer guidance

What flipped: longer useful task horizons

- Frontier agents can sustain multi-hour, well-scoped work
- Reliability still declines as tasks get longer and messier

Standards now have institutional homes: AAIF and A2A

The players you'll meet all semester

Layer	What it does	Examples
Orchestration	Wire agents into workflows	LangGraph, OpenAI Agents SDK, Google ADK
Protocols	Connect tools, agents, and commerce workflows	MCP (tools/data), A2A (agents), UCP (commerce)
Runtimes / SDKs	Build and run agent systems	OpenClaw, LangGraph, OpenAI Agents SDK, Google ADK
Models	The reasoning engine underneath	Gemini, GPT, Claude, Gemma 4 (local)
Standards	Keep the above interoperable	Agentic AI Foundation (Linux Foundation)

Today's lens: Karpathy's Software 3.0

From "Software Is Changing (Again)"

- Andrej Karpathy, YC AI Startup School, June 2025
- "Software 3.0" is the framework introduced in that talk

A useful model for the semester — not a doctrine

- We'll use it to read the landscape, then pressure-test it

Software 1.0 — you write the code

Explicit instructions, written by humans, in a programming language

- The artifact is source code
- The computer does exactly what you wrote

Seventy years of software lived here

```
int max(int a, int b) {  
    // explicit logic, by hand  
    if (a > b) {  
        return a;  
    }  
    return b;  
}
```

Software 2.0 — curate the data

Karpathy's 2017 framing (Medium, Nov 11, 2017)

- The program is a neural network; the source is the dataset
- You don't write the logic — you shape the data and train

An image classifier, the two ways

- 1.0: hand-coded edge and feature detectors that mostly failed
- 2.0: label thousands of images, train a CNN, weights are the program

Why it matters here: the code stopped being readable

- You can't step through a weight matrix line by line
- That non-readability is the bridge to 3.0

Software 3.0 — you specify behavior

The LLM is the runtime. English is the syntax.

- The artifact expands: instructions, context, tools, policies, and evals
- Prompts and specs are central — but they are not the whole system
- Karpathy: the LLM behaves like a new kind of operating system

```
# system prompt (this IS the program)
```

```
You are a scheduling agent.
```

```
Read the user's calendar.
```

```
Propose 3 conflict-free slots.
```

```
Rank them by the user's stated  
preferences.
```

```
Never book without approval.
```

Three eras, side by side

	Software 1.0	Software 2.0	Software 3.0
Artifact	Source code	Trained weights	Instructions, context, tools, policies, evals
You write	Logic, by hand	Data + training loop	Intent; the model fills the logic
Debugging	Read the stack trace	Inspect data and metrics; retrain; validate	Re-read output, tighten wording, constrain, re-run, verify
Version control	Git on .c/.py	Git-LFS on weights + data lineage	Git on prompts and configuration — e.g., AGENTS.md, SOUL.md, skills

Karpathy's claim: the foundation is shifting

"Software has not changed much on such a fundamental level for 70 years — then it's changed about twice quite rapidly in the last few years."

- — Karpathy, YC AI Startup School, June 2025

His framing is directional, not a percentage

- Claim: 3.0 is poised to rewrite large parts of the existing codebase
- He puts no number on it — and neither will we

Why the discipline matters

- Unsourced stats are how hype enters a technical talk
- Same rule binds your assignments: cite it or cut it

Course model: all three layers persist

"Everything is 3.0 now" is the easy mistake

A real agent runs all three layers at once:

- 3.0 — the prompts and specs you author
- 2.0 — the foundation model doing the reasoning
- 1.0 — the OS, runtime code, network, and optional container

The skill is knowing which layer you're in

- A bug can live in any of the three — they fail differently
- Today's demo traces all three across the repo and runtime workspace

Agents make Software 3.0 concrete

In an agent, the spec is the program — the English you write is what runs

Generic architecture: instructions + tools + state/memory + runtime

- OpenClaw tools — SKILL.md files describe capabilities
- OpenClaw memory — MEMORY.md and dated Markdown files
- OpenClaw behavior — workspace instructions set goals and constraints

That is why this course builds agents

The Claw Phenomenon



OpenClaw — and how fast this space moves

An open-source autonomous agent you self-host

- Messaging apps are the interface; memory is local Markdown

Project lineage began Nov 24, 2025 with Peter Steinberger

- Warelay → Clawdis → Clawdbot → Moltbot → OpenClaw
- The Clawdbot rename followed an Anthropic naming objection

386,000+ GitHub stars as of Aug 11, 2026

- MIT licensed; developed under the nonprofit OpenClaw Foundation
- Anchor on patterns here, not names — they move monthly

"Claw" is a family, not one project

Project	Type	What it optimizes for
OpenClaw	Implementation	Features, reach; permissive by default
NanoClaw	Implementation	Small, single-process design; container isolation
NemoClaw	Reference stack	NVIDIA OpenShell safety; hosted or local inference routes
IronClaw	Implementation	Rust + WASM sandbox; credential protection
ZeroClaw	Implementation	Small, single-binary Rust runtime
ClawHub	Registry	Skills and plugins — not an agent itself

OpenClaw, structurally

Two structures you'll see during the demo:

- Code repository — gateway, apps, extensions, skills, packages
- Runtime workspace — instructions, identity, user context, memory

Channels and extensions provide the chat-app front door

Docker deployment and tool sandboxing are optional

```
openclaw/      ~/.openclaw/workspace/  
src/           AGENTS.md  
skills/        SOUL.md  
extensions/    USER.md  
apps/          IDENTITY.md  
packages/      MEMORY.md  
Dockerfile     memory/YYYY-MM-DD.md
```

Workspace files: the agent's brain is English

SOUL.md, USER.md, IDENTITY.md — version-controlled Markdown

- Persona, user context, and identity — not task state by themselves
- Persistent memory: MEMORY.md + dated memory/YYYY-MM-DD.md files
- This is Software 3.0 you can read in a text editor

```
# Illustrative SOUL.md
```

```
Be concise.
```

```
Push back when something seems wrong.
```

```
Do not just execute.
```

```
Apologize only for actual mistakes.
```

Skills: capability as a Markdown spec

A skill folder requires SKILL.md; supporting files are optional

- SKILL.md is Markdown with optional YAML frontmatter
- ClawHub distributes skills and plugins

Agent Skills is a portable format supported across multiple products

```
---  
name: weather  
description: Get a 3-day forecast.  
---  
Call wttr.in for the city.  
Return plain text.
```

Gateway, channels, and optional isolation

The Node.js/TypeScript gateway normally runs on the host

- Channels connect Telegram, WhatsApp, Discord, Signal, and others

Docker deployment and tool sandboxing are optional

- When enabled: dropped capabilities, restricted network, read-only root

```
# Optional Docker deployment
services:
  openclaw-gateway:
    cap_drop: [NET_RAW, NET_ADMIN]
    security_opt:
      - no-new-privileges:true
```

Live: Inside OpenClaw



Live demo: inside OpenClaw

LIVE DEMO

Switch — between the OpenClaw repository and a configured runtime workspace

Compare — repo code (src/, skills/, extensions/) with ~/.openclaw/workspace

Inspect — one workspace instruction file, persistent memory, and one SKILL.md

Note — the Node.js/TypeScript gateway is required; Docker and sandboxing are optional

Time budget — ~10 minutes, including the three-layer wrap-up

What we just saw

English files configured behavior — SOUL.md, memory, skill specs

TypeScript implemented the gateway, routing, sessions, and tools

The model reasoned; optional Docker or sandbox policy isolated execution

Why that was Software 3.0, layered

Layer	In OpenClaw	Era
What you author	SOUL.md, USER.md, SKILL.md	3.0 — English specs
What reasons	The LLM the agent calls	2.0 — trained weights
What runs it	Node.js/TypeScript gateway; optional Docker/sandbox	1.0 — explicit code

The Agent Manager



The job changes: from typing to directing

If the spec is the program, your work moves up a level

- Specify — write the intent precisely
- Verify — check what the agent produced
- Ship — take responsibility for the result

But the manager still does plenty by hand

- Debugging the model's output when the spec didn't take
- The judgment calls an agent can't make — what's worth building

When to specify—and when to type

PAPA'S RULE

In 3.0, your job is to specify, verify, and ship — not to type. If you're typing, you're doing 1.0 inside a 3.0 stack.

When typing IS the right move:

- Debugging the model's output — the part it got wrong
- The narrow, exact change a spec would only fumble

Type deliberately — know when you've dropped to 1.0

Standards arrived early: the AAILF

Agentic AI Foundation — Linux Foundation, Dec 9, 2025 (aaif.io)

- A neutral home for agent standards, against vendor lock-in

Three founding-contribution projects:

- MCP (Anthropic) — tool connectivity
- goose (Block) — local-first agent framework
- AGENTS.md (OpenAI) — repo-level agent instructions

AGENTS.md began as an internal solution for Codex

- Released publicly in Aug 2025, adopted broadly, then donated

A2A is a separate Linux Foundation project, not a founding AAILF contribution

Where we are

Software shifted twice, fast — we're living in the third shift

Agents make 3.0 concrete: natural-language specifications shape behavior

You saw the code repository and runtime workspace; next you start building

Next - Working Inside Software 3.0

This lecture showed what 3.0 looks like; next comes working inside it

- Agentic coding tools — Claude Code and OpenAI Codex
- AGENTS.md and CLAUDE.md — telling agents how your repository works
- Vibe coding as method — and a live build of a small agent